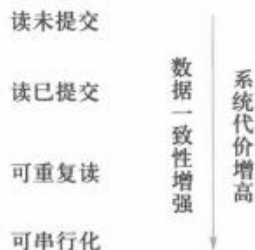


表 12.1 事务隔离级别与数据不一致性的关系

事务隔离级别	数据不一致性			
	丢失修改	脏读	不可重复读	* 幻读
读未提交	否	是	是	是
读已提交	否	否	是	是
可重复读	否	否	否	是
可串行化	否	否	否	否

事务隔离级别并不是越高越好，它与数据一致性以及系统代价的关系如图 12.3 所示。数据库管理系统通常都提供了设置事务隔离级别的方法，在实际中，用户应根据应用的特点和需求选择合适的事务隔离级别。

二维码所示内容通过实验来演示 4 种事务隔离级别下丢失修改、脏读、不可重复读等数据不一致的情况，读者可以直观地感受各种隔离级别在事务中的作用。



扩展阅读：隔离级别实验

图 12.3 事务隔离级别与数据一致性及系统代价的关系

12.3 封锁

封锁是实现并发控制的一种非常重要的技术。所谓封锁，就是事务 T 在对某个数据对象操作之前，先向系统发出请求，对其加锁。加锁后事务 T 就对该数据对象有了一定的控制，在事务 T 释放它的锁之前，其他事务不能更新或读取此数据对象。例如，在例 12.1 中，事务 T_1 要修改 A，若在读出 A 前先锁住 A，其他事务就不能再读取和修改 A 了，直到 T_1 修改并写回 A 后解除了对 A 的封锁为止。这样，就不会丢失 T_1 的修改。

确切的控制由封锁的类型决定。基本的封锁类型有两种：排他型锁(exclusive lock，简称 X 锁)和共享型锁(shared lock，简称 S 锁)。

排他型锁又称为写锁(write lock)。若事务 T 对数据对象 A 加上 X 锁，则只允许 T 读取和修改 A，其他任何事务都不能再对 A 加任何类型的锁，直到 T 释放 A 上的锁为止。这就保证了其他事务在 T 释放 A 上的锁之前不能再读取和修改 A。

共享型锁又称为**读锁**(read lock)。若事务 T 对数据对象 A 加上 S 锁, 则事务 T 可以读 A 但不能修改 A , 其他事务只能再对 A 加 S 锁, 而不能加 X 锁, 直到 T 释放 A 上的 S 锁为止。这就保证了其他事务可以读 A , 但在 T 释放 A 上的 S 锁之前不能对 A 做任何修改。

排他型锁与共享型锁的控制方式可以用图 12.4 所示的相容矩阵 (compatibility matrix) 来表示。

T_1	T_2		
	X	S	—
X	N	N	Y
S	N	Y	Y
—	Y	Y	Y

注: Y=Yes, 相容的请求; N=No, 不相容的请求。

图 12.4 封锁类型的相容矩阵

在图 12.4 所示的封锁类型的相容矩阵中, 最左边一列表示事务 T_1 已经获得的数据对象上锁的类型, 其中横线表示没有加锁。最上面一行表示另一事务 T_2 对同一数据对象发出的封锁请求。 T_2 的封锁请求能否被满足用矩阵中的 Y 和 N 表示, 其中 Y 表示事务 T_2 的封锁要求与 T_1 已持有的锁相容, 封锁请求可以满足; N 表示 T_2 的封锁请求与 T_1 已持有的锁冲突, T_2 的请求被拒绝。

12.4 封锁协议

在运用 X 锁和 S 锁这两种基本封锁类型对数据对象加锁时, 还需要约定一些规则。例如, 何时申请 X 锁或 S 锁、持锁时间、何时释放等。这些规则称为**封锁协议**(locking protocol)。对封锁方式制定不同的规则, 就形成了各种不同的封锁协议。本节介绍三级封锁协议。对并发操作的不正确调度可能会带来丢失修改、不可重复读和脏读等数据不一致性问题, 三级封锁协议分别在不同程度上解决了这些问题, 为并发操作的正确调度提供了一定的保证。不同级别的封锁协议达到的数据一致性级别是不同的。

1. 一级封锁协议

一级封锁协议是指事务 T 在修改数据 R 之前必须先对其加 X 锁, 直到事务结束才释放。事务结束包括正常结束 (COMMIT) 和非正常结束 (ROLLBACK)。

一级封锁协议可防止丢失修改, 并保证事务 T 是可恢复的。例如图 12.5(a) 使用一级封锁协议解决了图 12.2(a) 中的丢失修改问题。

图 12.5(a) 中事务 T_1 在读 A 进行修改之前先对 A 加 X 锁, 当 T_2 再请求对 A 加 X 锁时被拒绝, T_2 只能等待 T_1 释放 A 上的锁后获得对 A 的 X 锁, 这时它读到的 A 已经是 T_1 更新过的值 15, 再按此新的 A 值进行运算, 并将结果值 $A=14$ 写回到磁盘。这样就避免了丢失 T_1 的修改。

在一级封锁协议中, 如果仅读数据而不对其进行修改是不需要加锁的, 所以它不能保证可重复读和不脏读。